
MLOps Experimental Learning Assignment 01

End-to-end model development, CI/CD and deployment using the UCI Cleveland Heart Disease data

Course	AIMLCZG523 — Machine Learning Operations
Submission	Student name / ID: MAHENDRA SINGH / 2025AF05023
Environment	BITS Prayogshala virtual lab; persistent path ~/workspace
Repository	https://github.com/2025af05023-MS/heart-disease-mlops-assignment
Date	4 October 2026

Abstract

I used the Cleveland records to follow one model from the original UCI file through a working API in the lab. I checked and cleaned the data, compared two pipelines with the same split, logged the experiments in MLflow, and packaged the selected model for FastAPI. GitHub Actions repeats the checks; Docker, Minikube and API metrics show what happened after training. This is a classroom demonstration, not a clinical tool.

Status note

I repeated preparation and training in the Prayogshala VM. Eight tests and Ruff passed there. The Docker image served a real prediction; Minikube ran two ready API pods, and the service returned a prediction and Prometheus metrics. The GitHub Actions workflow passed on the first public push; its screenshot is included on page 8. A short silent lab screen recording is supplied at [demo/pipeline_walkthrough.webm](#).

Key design choice

All imputation, one-hot encoding and scaling live inside each scikit-learn pipeline. The 20% hold-out set is used only after five-fold cross-validation selects the model family. This avoids using test data during feature preparation or tuning.

Problem statement and dataset

The task is binary classification: given 13 routinely recorded fields from the processed Cleveland data, predict whether the original UCI outcome indicates disease presence. The historical outcome **num = 0** is mapped to 0, while values 1–4 map to 1. This follows the dataset documentation. The API returns a model score and class label, not a medical diagnosis.

The assignment brief refers to “14+ features,” while the official processed Cleveland release has 14 columns total: 13 predictors plus the outcome. I use the source schema as published rather than inventing an extra predictor.

Source	UCI Machine Learning Repository, Heart Disease (processed Cleveland)
Records	303 rows; 13 features and one outcome
Target balance	164 absence (54.1%); 139 presence (45.9%)
Missing inputs	4 missing ca values; 2 missing thal values
Attribution	Janosi et al. (1989), DOI 10.24432/C52P4X; CC BY 4.0

Acquisition and cleaning

The source URL and a download script are in `src/heartlab/data.py`. The original file is saved verbatim. The cleaner checks the 14-field schema and target range, turns UCI question marks into missing values, converts columns to numeric values and writes a CSV with a binary target. It does not fill missing values before the train/test split. A copy of the cleaned CSV is included in the project.

Practical data caution

The dataset has no patient identifiers in the processed file, but the features still describe health. For this classroom system, examples should come from the published dataset and API logs should omit raw input values. The sample is small and historical, so modern clinical or population performance cannot be inferred from these scores.

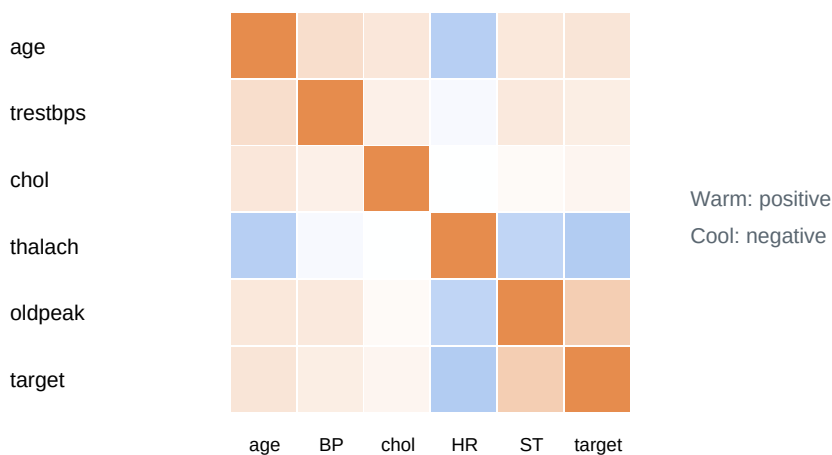
Exploratory data analysis

The EDA script produces three figures: selected feature histograms by target, a class-balance chart, and a Pearson correlation heatmap. I use the heatmap as a screening view only because integers that code categories do not have a meaningful numeric distance. The code saves high-resolution PNG files for the report and MLflow.

Target balance (303 records)



Selected Pearson correlations



Checks from the cleaned file

Check	Result / interpretation
Rows and fields	303 rows; no dropped records; 13 input features
Missing values	ca: 4; thal: 2; median/mode imputation deferred to pipeline
Outcome	164 negative, 139 positive; stratification preserves this mix
Visual review	Age, cholesterol and maximum heart rate histograms are produced by eda.py

The class balance is reasonably close but not perfectly equal. I therefore retain stratified folds and report recall alongside accuracy. The visual EDA is descriptive and does not prove causal effects or justify medical use.

Preprocessing and feature engineering

I divided the 13 inputs into five numerical variables (age, resting blood pressure, cholesterol, maximum heart rate and oldpeak) and eight coded categorical variables. The numerical branch fits a median imputer and standard scaler; the categorical branch fits a most-frequent imputer and one-hot encoder. Unknown categories at inference are ignored so the service can fail gracefully rather than crashing.

Stage	Implementation	Reason
Split	80/20 stratified, random_state=42	Keeps a single untouched test set
Numerical	Median imputer + StandardScaler	Handles missing values and LR scale sensitivity
Categorical	Mode imputer + OneHotEncoder	Avoids false numerical ordering
CV	StratifiedKFold(5), shuffled, seed 42	Comparable and repeatable model selection
Artifact	joblib dump of complete Pipeline	Same transformations in training and serving

Why the pipeline matters

A tempting shortcut is to calculate medians, scales or encodings once on the whole CSV before cross-validation. That would leak information from validation folds. Here scikit-learn refits each transformer within every fold. The serialized final pipeline contains its fitted transformations, so a fresh JSON request follows the same steps as training.

Schema contract

The API expects the 13 named fields and rejects unknown fields and out-of-range values with a validation error. ca and thal can be null because the original data has missing values. The training CSV and request model are deliberately small and easy to inspect.

Models, tuning and evaluation

I wanted to see whether a more flexible model helped enough to justify its complexity, so I compared logistic regression with random forest under the same five stratified folds. For logistic regression I tried C values 0.1, 1.0 and 10.0. For the forest I tried 100/200 trees, depth 3/unrestricted and leaf size 1/3. The comparison used the same training split throughout.

Metric	Why it is included
Accuracy	Overall correctness; can hide class-specific errors
Precision	How often a predicted positive is truly positive
Recall	How many positive cases are found
ROC-AUC	Ranking discrimination across thresholds; selection metric

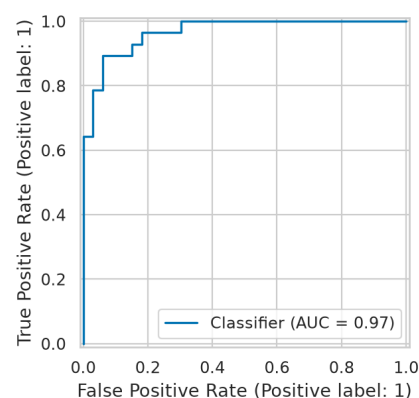
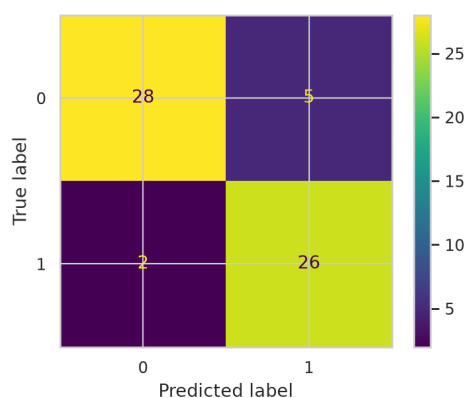
Selection rule

The model with the higher **mean cross-validation ROC-AUC on the training split** is selected. Only then is its predicted probability evaluated on the held-out test split. The classification threshold is fixed at 0.5, and a confusion matrix plus ROC curve are saved. This separation keeps the test estimate from steering the selection.

Observed five-fold CV results

Candidate	ROC-AUC	Accuracy	Precision	Recall
logistic_regression	0.902	0.847	0.877	0.783
random_forest	0.899	0.814	0.815	0.774

Selected: logistic_regression. Held-out metrics: accuracy 0.885, precision 0.839, recall 0.929, roc_auc 0.966.



Left: held-out confusion matrix. Right: held-out ROC curve. Both were generated by the completed training run.

Experiment tracking and packaging

MLflow is configured with a local SQLite tracking database, `mlflow.db`. Each model family has a run with its best grid parameters, seed, training size, mean CV accuracy, precision, recall and ROC-AUC, plus the EDA plots. A final selection run records the chosen family, held-out metrics, serialized pipeline and diagnostic figures.

Artifact	Location / purpose
Raw source	<code>data/processed.cleveland.data</code> — exact UCI download
Clean CSV	<code>data/cleveland_clean.csv</code> — repeatable modelling input
Fitted model	<code>artifacts/model.joblib</code> — full preprocessing + classifier
Metadata	<code>artifacts/metadata.json</code> — split, seed, CV scores, test scores
Visuals	<code>artifacts/eda</code> , <code>confusion_matrix.png</code> , <code>roc_curve.png</code>
Run history	Generated by training in each environment; local summary in <code>reports/MLFLOW_RUNS.md</code>

Clean-room repeat

Install dependencies from `requirements.txt`, run the data module, run lint and tests, then run training. A fixed seed and explicit search grid keep the experiment repeatable. Exact bit-for-bit metrics may still depend on dependency versions and platform; the requirements file uses bounded versions rather than a full lockfile, which is a limitation worth addressing in a larger project.

API and container

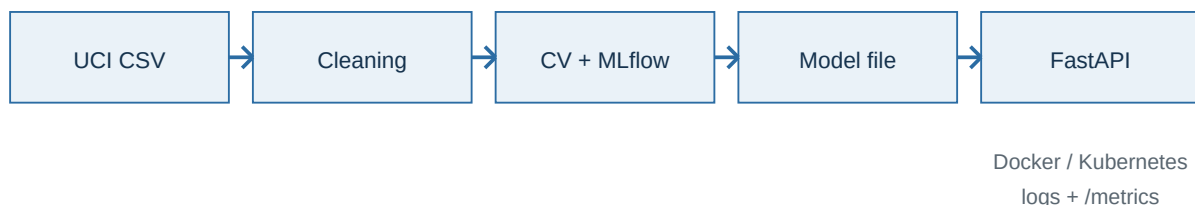
FastAPI loads the joblib pipeline on the first health or prediction request. POST /predict accepts the 13 feature names as JSON and returns the binary class, positive-class probability, a confidence value and model version. GET /health confirms that the model can be loaded; GET /metrics publishes Prometheus-format counters and a latency histogram.

Endpoint	Contract
POST /predict	200: prediction, confidence, probability_positive, model_version
GET /health	200 when model loads; 503 otherwise
GET /metrics	Prometheus text exposition
Validation	422 for malformed or out-of-range JSON
Logging	Event, status and duration only; no individual feature values

Docker package

The Dockerfile starts from python:3.11-slim, installs requirements, copies source and the trained model, then runs Uvicorn as a non-root user on port 8000. The model must be trained before docker build. A local smoke test uses the supplied sample_request.json. The CI workflow also builds and tests the image.

System architecture



Example request

```
curl -s -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" -d @sample_request.json
```

CI/CD workflow and tests

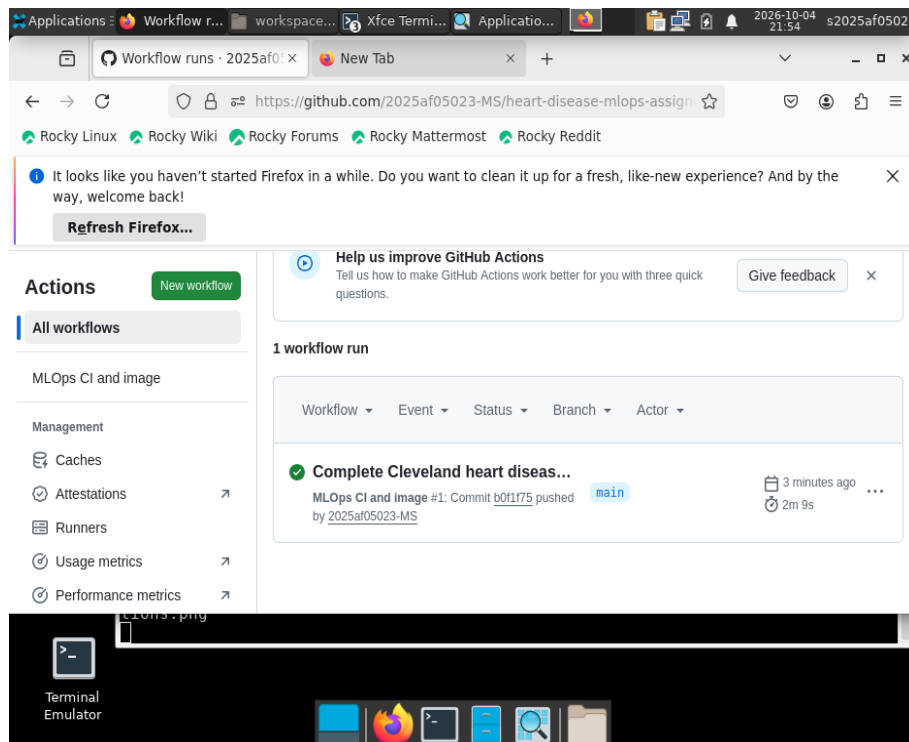
The GitHub Actions workflow runs on push, pull request and manual dispatch. It installs dependencies, runs Ruff and Pytest, downloads the public dataset, trains the candidates with MLflow, builds the Docker image, sends a sample request to the container, and uploads the logs, model, figures and run store as workflow artifacts. Any failing lint, test, training, build or smoke-test command fails the job.

Gate	Evidence from workflow
Lint	ruff check src tests
Unit tests	Cleaning, invalid target, pipeline missing values, metrics, API response and validation
Training	Two CV searches plus saved model and metadata
Container	docker build followed by a real HTTP prediction
Artifacts	training.log, cleaned CSV, model/plots and mlruns

Deployment handoff

The manifest is applied separately in the lab after the tested image is available. This is deliberate because the course lab may not expose credentials for an automatic GitHub-to-cluster deployment. The workflow does not claim a deployment it has not performed. A production setup would add image publishing, an environment approval, secret management and automated rollout to a managed cluster.

Successful GitHub Actions run



The public main-branch workflow for commit b0f1f75 completed successfully in 2 minutes 9 seconds. Screenshot captured inside the Prayogshala VM.

Kubernetes deployment and monitoring

The Kubernetes manifest defines a two-replica Deployment, a LoadBalancer Service on port 8000, readiness and liveness probes, resource requests and limits, and a non-root container security context. Prometheus scrape annotations point to /metrics. A Minikube deployment can expose the service through minikube tunnel; if tunnel cannot receive an external IP in the lab, port-forward is an honest local access path.

Check	Command / expected observation
Rollout	kubectl rollout status deployment/heartlab-api
Pods and service	kubectl get pods,service; two ready pods
Endpoint	curl /health and POST /predict through the exposed address
Logs	kubectl logs deployment/heartlab-api
Metrics	curl /metrics; request counter and latency histogram

Monitoring interpretation

The counter reveals successful, unavailable and failed requests. The latency histogram can show slowing predictions. These are operational metrics, not data-drift or clinical safety monitoring. A larger deployment should add availability alerts, distribution drift checks, retraining governance and review of model performance on newly labelled data.

Observed lab deployment

The first rollout failed because Kubernetes could not verify the image's named non-root user. Adding runAsUser: 1000 to the manifest fixed it. The second rollout reported 2/2 replicas available, and both pods were ready. The LoadBalancer external IP stayed pending in Minikube, so I used port-forward for the service request. The direct lab screenshot is saved at screenshots/kubernetes.png, with a command log in reports/LAB_VERIFICATION.md.

Verification, limitations and references

Item	Current evidence
UCI data	Verified locally and in lab: 303 rows; cleaned CSV included
Code checks	Ruff passed; eight Pytest tests passed in lab
Lab dependencies	Installed in persistent ~/workspace/.venv
MLflow and model	Two candidate runs and selection run logged in lab; model saved
API	Container and Kubernetes service returned ready and prediction
Docker/Kubernetes	Image built; two ready pods; direct lab screenshot saved
GitHub Actions	Passed on first public push; screenshot included
Demonstration video	Recorded in the Prayogshala VM as WebM

Limitations

The data is historical and limited to 303 Cleveland records. A single held-out split has high variance; cross-validation helps comparison but does not establish clinical transportability. The positive-class probability is not clinically calibrated. Missing values are imputed from training folds, but the training set is too small to guarantee stable estimates. The API is a teaching example and must not guide care.

What I would improve next

The first Kubernetes rollout taught me that a passing model test does not guarantee a running service: the missing numeric `runAsUser` blocked the pods until I fixed the manifest. With more time, I would lock dependencies, test calibration and external data, add drift checks, and publish the tested image through a gated deployment. I would also review threshold trade-offs before any real-world use.

References

1. Janosi, A., Steinbrunn, W., Pfisterer, M., and Detrano, R. (1989). Heart Disease [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C52P4X>.
2. BITS Pilani, AIMLCZG523 Assignment 01 instructions (supplied PDF).
3. BITS Pilani, Prayogshala Virtual Lab Manual (supplied PDF).